

A Simple k NN Search Protocol using Data Broadcasting in Wireless Mobile Environments

Kai-Yun Ho, Chuan-Ming Liu, and Chien-Hung Liu
 Department of Computer Science and Information Engineering
 National Taipei University of Technology
 Taipei 106, TAIWAN
 {s4618026, cmliu, cliu}@ntut.edu.tw

Abstract

Data broadcasting is an effective way to disseminate information to a large amount of mobile clients in wireless mobile environments. Many information services can use such a technique to serve the clients, including location-based service. The k nearest neighbor (k NN) search is one of the important location-based services. With such a search, the clients can get the points of interests around them. In this paper, we propose a k NN search protocol using data broadcasting. We consider how the server generates the broadcast schedule and how the client can efficiently execute the query process in terms of latency (time interval from issuing and terminating a query) and tuning time (time spent on listening to the broadcast). Without using an index in the broadcast, the proposed protocol uses some additional information for each broadcast data instead in order to save the tuning time and latency. The effectiveness of the proposed protocol will be verified by proofs.

1 Introduction

Emerging technologies on wireless communication, positioning systems, and networking make it possible for people to access information ubiquitously. In such a wireless mobile environment, the bandwidth of the downlink is much larger than the bandwidth of the uplink [1, 13]. Under such an asymmetric environment, the traditional one-to-one communication between clients and servers is not an appropriate method to provide the information or services because of the bottleneck of the uplinks. Instead, data broadcasting provides an effective way to disseminate information to a large pool of mobile clients.

In different information services, the location-based service (LBS) is one of the important services and provides the information related to the positions of the

mobile clients. The k nearest neighbors (k NN) search is one of the query types in the location-based service. A mobile client can have the query like "where is the nearest gas station?" or "please give me five nearest book stores". In general, the k NN search at a query point q is to find k objects closest to q in a given data set. In this paper, we consider how to provide the k NN search using data broadcasting in wireless mobile environments. This problem has been studied by [6, 19, 28].

When applying data broadcasting to provide information services, two cost measures are usually considered. The *latency* (i.e., the time elapsed between issuing and termination of the query) indicates the Quality of Service (QoS) provided by the system and the *tuning time* (i.e., the amount of time spent on listening to the channel) represents the power consumption of mobile clients. In general, when the broadcast consists of only data without an index, the two cost measures are equivalent. In contrast, broadcasting data with an index is an efficient approach in terms of energy consumption [8, 9, 13, 14, 18, 19, 28] since the index allows a mobile client to tune into a broadcast only when data of interest and relevance is available.

When designing the data broadcasting protocols, two aspects are generally discussed. From the perspective of a server, how to organize the data in the broadcast in order that the clients can access the information efficiently in terms of the latency and tuning time is challenging. From the viewpoint of a mobile client, how to receive the information in an energy-efficient manner (i.e., with fewer tuning time) raises an important issue since the energy is a scarce resource for a mobile client.

Our proposed protocol for the k NN search will consider how the server schedules the broadcast and how the client efficiently executes the query processing on the corresponding broadcast. In our design, we will not include an index on the data set. Instead, we add some additional information with each data point to al-

low the selective tuning in order to reduce the tuning time (energy consumption). For simplicity, we assume that each data point corresponds to one packet in the broadcast. The additional information is derived from the Voronoi diagram composed by the data set. More details about how to derive the addition information and scheduling is presented in Section 3.1. On the client side, we provide, in Section 3.2, a mechanism to efficiently execute the query processing in terms of the latency and tuning time according to the broadcast.

Besides the presentation and analysis of the proposed protocol in Section 3, we provide the experimental results in Section 4. Section 5 concludes this paper. In the next section, we will give the related works and some preliminaries.

2 Preliminaries

Topics related data broadcasting have been studied for many years. The papers about data broadcasting without an index focus on how to schedule the broadcast in order to achieve a shorter latency [2, 4, 10, 11, 12, 13, 17, 20, 22, 25, 26, 27]. By adding an index in the broadcast can reduce the tuning time; therefore, leading a less power consumption [6, 7, 8, 13, 14, 18, 19, 28].

The nearest neighbor (NN) search problem has been studied for many decades and many solutions has been proposed. One approach to solve the NN search problem for a given data set is to use an index structure, like R-trees, k-d-trees, or Quad-trees [24, 23]. The first NN search algorithm on an R-tree [21] followed the branch-and-bound algorithmic design pattern and could be generalized to the k NN search ($k > 1$). This algorithm searches the R-tree in a depth-first fashion and prunes the irrelevant nodes during the traversal. Based on this algorithm, the authors in [6] and [19] proposed different approaches for the k NN search using broadcast R-trees in wireless broadcast environments.

The other approach to find the k NN for a given query point in a data set is to use the Voronoi diagram [3, 28]. A Voronoi diagram for a given data set can be constructed by computing the perpendicular bisectors of the line segments between the pairs of data points. Each data point is then associated with a cell in the Voronoi diagram [5]. The nearest data point of a query point q is the data point whose associated cell contains q . Hence, one can find the nearest neighbor by examining the cells in the Voronoi diagram. Using the Voronoi diagram can only help one to find the NN for a query point. One can not find the k NN simply for an arbitrary $k > 1$. To extend the finding for k NN, one

can consider the order- k Voronoi diagram [16]. However, the value of k is fixed. In [15], the authors proposed an approach for k NN queries in spatial network databases using order-1 Voronoi diagram. The search first precomputes the possible data points around each cell and then filter the considered cells and the neighboring cells until the result is done. In wireless broadcast environments, the authors in [28] discussed some efficient NN search protocols, including the solution-based and object-based approaches, and proposed a hybrid one. However, their solutions are only for the NN search. Our proposed approach will use the order-1 Voronoi diagram and consider the neighbors of each site, when generating the broadcast schedule.

In the following of this section, we give the terminology and notations used in this paper. Given two data points p and q in a plane, we denote the distance between p and q as $dist(p, q)$ and

$$dist(p, q) = \sqrt{(p_x - q_x)^2 + (p_y - q_y)^2}.$$

Let $P = \{p_1, p_2, \dots, p_n\}$ be the set of n distinct data points in the plane. These points are the *sites*. In the following of this paper, the *data points* or *sites* are used interchangeably. For each site $p_i \in P$, $i = 1, 2, \dots, n$, we define the *Voronoi cell* associated with p_i as the area

$$v(p_i) = \bigcap_{j \neq i} \{r | dist(r, p_i) \leq dist(r, p_j)\}, \forall j \leq n,$$

where $r \in R^2$. Using the Voronoi cells, the Voronoi diagram of P now can be defined as

$$VD(P) = \{v(p_i) | i = 1, 2, \dots, n\}.$$

Besides, in the data set P , the k nearest neighbors (k NN) of a given query point q is the set

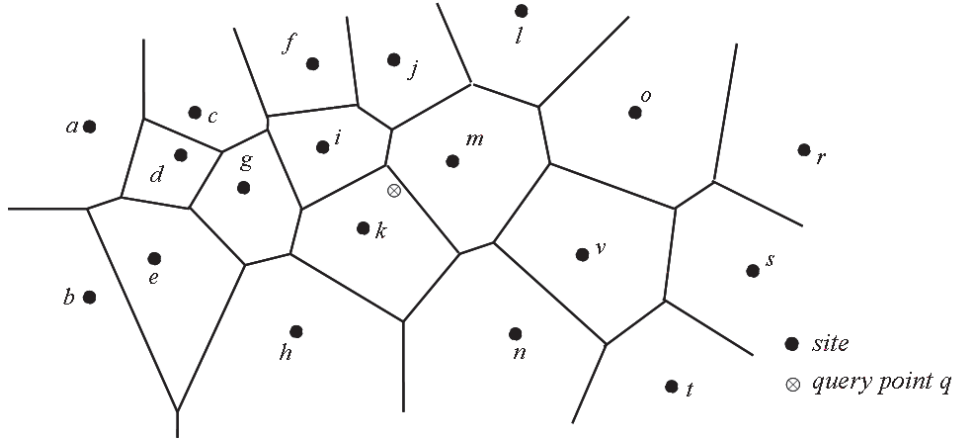
$$kNN(q) = \{p_i \in P | dist(q, p_i) \leq dist(q, o)\},$$

where $o \in P$ is the k th nearest neighbor of q . Figure 1 shows a Voronoi diagram for a data points on the plane and the 3NN of the query point q is $3NN = \{i, k, m\}$.

If two sites shares the same boundary of the corresponding Voronoi cells, these two sites are *1-order neighbors* to each other. For a site p_i , if the associated Voronoi cell has m boundaries, then p_i has m 1-order neighbors, say sites $u_1, u_2, \dots, u_m$. We define the set of the 1-order neighbors of p_i as

$$Adj(p_i) = \{u_1, u_2, \dots, u_m\}.$$

For instance, in Figure 1, the 1-order neighbors of d is $Adj(d) = \{a, c, e, g\}$.


 Figure 1: A Voronoi diagram for a set of data points on the plane and a query point q .

3 Proposed k NN Search Protocol

A straightforward approach for the k NN search in wireless broadcast environment is to have the server broadcast all the data points sequentially and the clients filter all the data points by always tuning into the broadcast. However, this approach is too naive and less efficient in terms of latency and tuning time since all the data points should be examined. So, some papers use an index in the broadcast to improve the performance on the latency and tuning time. In this section, we present our k NN search protocol which do not use an index but add some additional information in each site in the broadcast. Using the proposed approach, the client can perform the selective tuning to save the energy and will terminate the query processing when necessary, thus leading to a shorter latency. We now describe our approach in two parts: on the server side, we explore how to generate the broadcast; on the client side, we design an efficient query processing for the k NN search.

3.1 Broadcast Schedule on the Server

Our k NN search protocol for wireless broadcast environments is based on the NN search using Voronoi diagram. For a given data set $P = \{p_1, p_2, \dots, p_n\}$, the corresponding Voronoi diagram $VD(P)$ is first constructed. Note that a broadcast consists of a sequence of packets. For simplicity, the packet ID is the position (or address) of that packet in the broadcast. We will assign each site to one packet in the broadcast. For the time being, we simply assign the sites to the packets randomly. With such a broadcast, a straightforward solution for search the k NN is to scan all the packets in the broadcast.

	position	location
o	2	(o_x, o_y)
m	8	(m_x, m_y)
v	18	(v_x, v_y)
l	3	(l_x, l_y)
s	6	(s_x, s_y)
r	19	(r_x, r_y)

Figure 2: The data structure of a packet in the broadcast.

The above approach consumes too much energy and results in a long latency. In order to improve the efficiency for searching, our broadcast schedule will add additional information in each packet. The additional information is derived from the Voronoi diagram $VD(P)$. For each site p , we include p 's 1-order neighbors, $Adj(p)$, into the corresponding packet. As shown in Figure 2, the packet for site o contains o 's location and position in the broadcast as well as the location and position of each site in $Adj(o)$. The Voronoi diagram $VD(P)$ can be constructed in $O(n \log n)$ for data set $P = \{p_1, p_2, \dots, p_n\}$. The assignment for sites to the positions in the broadcast can be done in $O(n)$. Hence, the broadcast schedule can be generate in $O(n \log n)$ time. Figure 3 demonstrates a schedule for the data points (sites) given in Figure 1.

3.2 Client Query Processing

With the broadcast generated in the previous subsection, our query processing can start at an arbitrary time instance. There are two data structures used during the query execution. D -list is a sorted list using the

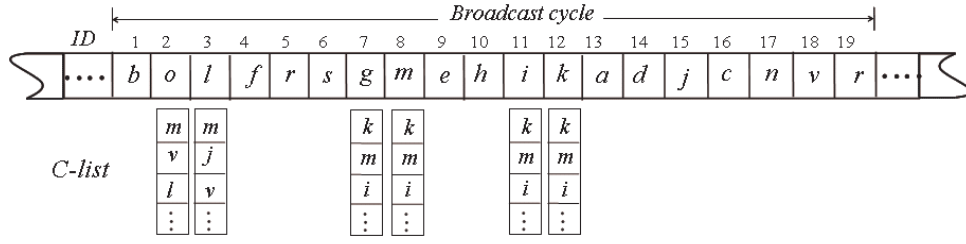


Figure 3: The example of 3NN query process.

position of a site as the key to store the positions the query processing can skip in an increasing order. *C-list* is a sorted list with the distance to query point q as the key and stores the candidate sites in an non-decreasing order. The high-level description of the algorithm for the query processing is shown in Figure 4.

Initially, all the lists are empty and a packet is received from the broadcast. The query process will examine all the information contained in the packet. Suppose the site in the received packet is p . Then site p is marked to indicate that p has been examined and the distance between p and q is also computed. For each site u in $Adj(p)$, the set of 1-order neighbors of p , we compute the distance between u and q . All the sites, including p and the 1-order neighbors of p , are inserted into *C-list* and all the sites in *C-list* are sorted according to their distances to q in a non-decreasing order. During the insertion, if a site is already in *C-list*, this site is ignored. In *C-list*, we decide the first k sites as the candidates for the k NN of q . If there are more than k sites in *C-list*, we insert the positions of the rest sites into *D-list* and then delete the them from *C-list*. If all the k sites in *C-list* are marked, the query process stops and these k sites is the k NN of q . Otherwise, the process continues and the client needs to access the next packet. The next packet to be received depends on *D-list*. If the position of the next packet is in *D-list*, the client can be in sleep mode and skip the next packet.

Consider the broadcast schedule in Figure 3 and suppose that we have a 3NN query at the query point q . Assume that the query starts at the position of site o . After receiving o , we mark o and increase the counter for the number of marked nodes by one. The 1-order neighbors of o are m, l, r, s , and v . We then insert all these sites, including o , into *C-list*. According to their distances to q , the 3 candidate sites are m, v , and l . The other sites can be deleted but their positions should be recorded for selective tuning. In this case, the positions 5 and 6 can be skipped since sites r and s are impossible to be included in the final result. Then we check whether all the candidate sites are marked. Since no

k NN-Query-VD(q)

```

1  count  $\leftarrow$  0;
2   $\triangleright$  count: count the number of marked sites.
3  while count  $\neq$   $k$ 
4      do receive a packet of site  $p$ ;
5      mark site  $p$ ;
6      count++;
7      compute  $dist(p, q)$ ;
8      for each  $u$  in  $Adj(p)$ 
9          do compute  $dist(u, q)$ 
10     insert  $p$  and each  $u$  into C-list;
11     decide the candidates in C-list;
12     if  $k < |C-list|$ 
13         then insert the positions of
14             the rest sites into D-list;
15         for each of the rest sites,  $u$ 
16             do if  $u$  is marked
17                 then count - -;
18                 delete  $u$  from C-list
19     Use D-list to compute the next
20     packet to receive

```

Figure 4: A high-level description of the k NN query processing on the client.

candidate site is marked at this time instance, the process continues to receive the next packet. The next packet can be determined by comparing the current position with the positions stored in *D-list*. Because position 3 isn't recorded in *D-list*, this position must be examined. When l (packet 3) is received, we mark l and insert sites j and f , into *C-list* only since i, o , and m have been inserted. According to the distance to q , we now keep m, j , and v in *C-list* and store positions 4, 5, and 6 in *D-list*. Then the next packet to receive is the packet at position 7. The same process continues until the packet at 12 which contains site k . After receiving this site, we mark k and all the sites in *C-list* are marked and the process stops. The resulting 3NN

is k, m, i . In this example, the process experiences 6 and 11 packets in tuning time and latency respectively.

3.3 Correctness

In following, we show the correctness of the proposed search protocol. Recall that the set of data points (sites) is $P = \{p_1, p_2, \dots, p_n\}$ and the corresponding Voronoi diagram is $VD(P)$. We start with the following Lemma.

Lemma 1 *Let $p_i \in P$ be the nearest neighbor of query point q . Then q is in Voronoi cell $v(p_i)$ and the next nearest neighbor of q is in $Adj(p_i)$.*

Proof: According to the definition of Voronoi diagram, it is easy to conclude that q is in Voronoi cell $v(p_i)$. Let p_j be the second nearest neighbor of q . We need to show that $p_j \in Adj(p_i)$. Assume that this is not true. We consider the line $\overline{qp_j}$. This line must cross a Voronoi cell which is next to $v(p_i)$, say $v(p_k)$ where p_k is the corresponding site. Then, p_k must be in $Adj(p_i)$. Let the two intersections of $\overline{qp_j}$ and $v(p_k)$ be a and b , respectively. Any point r on ab is in $v(p_k)$ and hence $dist(rp_k) \leq dist(rp_j)$. Then, $dist(q, r) + dist(r, p_k) \leq dist(q, r) + dist(r, p_j)$. Using the triangular inequality, we can derive $dist(q, p_k) \leq dist(q, r) + dist(r, p_k) \leq dist(q, r) + dist(r, p_j)$. This conclusion contradicts to our assumption. Hence, the next nearest neighbor of q is in the 1-order neighbors of p_i , $Adj(p_i)$. $\square$

The following lemma can be proved by simply extending the result in Lemma 1. We therefore describe lemma without giving a proof.

Lemma 2 *Suppose the kNN of a query point q is $\{p_1, p_2, \dots, p_k\}$. The $(k+1)$ th nearest neighbor of query point q must be in $\bigcup_{i=1}^k Adj(p_i)$.*

Theorem 3 *Given a kNN query at point q . If all the k sites are marked in C -list, those k sites in C -list are the kNN of q .*

Proof: Let the resulting kNN be $\{p_1, p_2, \dots, p_k\}$ and p_k be the k th nearest neighbor of q . We suppose that the query process starts and stops at position x and y , respectively. Let F be the set of the sites from position x to position y on the broadcast and B is the set of the rest sites from position $y+1$ to position z , where $z-x+1$ is the total number of packets in the broadcast cycle. Our proof consists of two parts:

- (1) When the query processing stops, the kNN of q in set F must be the k sites in C -list; and
- (2) For each site p_i in B , $dist(q, p_k) \leq dist(q, p_i)$.

For part (1), in our algorithm, a site is marked when it is received and the C -list stores the candidate sites according to their distances to q and arrange them in a non-decreasing order. During the execution of the algorithm, we delete the sites which are impossible in the resulting kNN since their distances to q are longer than the distance between the current k th nearest site and q . The algorithm thus will examine all the possible sites and ignore the sites which are impossible in the result. Hence, when the algorithm stops, the k sites kept in C -list is the kNN of q in F .

In order to make sure that no more sites in B should be examined, we claim part (2). We assume that $dist(q, p_k) \geq dist(q, p_i)$, $p_i \in B$, and argue this by contradiction. Then, p_k should be replaced by p_i in the resulting $kNN = \{p_1, p_2, \dots, p_k\}$ of q . Let the 1-order neighbors of the $(k-1)NN$ is $\{u_1, u_2, \dots, u_m\}$ for some integer m . Since $p_i \in B$, p_i does not belong to the kNN of q in set F and $p_i \notin \{u_1, u_2, \dots, u_m\}$ according to Lemma 2. We also know that $p_k \in \{u_1, u_2, \dots, u_m\}$. Since $dist(q, p_i) \leq dist(q, p_k)$, there exists at least one site u_j in $\{u_1, u_2, \dots, u_m\}$ with $dist(q, u_j) \leq dist(q, p_k)$, where $j \leq m$. Then, sites u_j and p_k are both in $\{u_1, u_2, \dots, u_m\}$ and $dist(q, u_j) \leq dist(q, p_k)$. According to our algorithm, u_j should be examined. Then, consider two cases. If u_j comes earlier than p_k in the broadcast, p_k will never be in the result. This contradicts to the conclusion in part (1). On the other hand, if u_j comes later than p_k in the broadcast, the process will not stop at p_k . This also contradicts to our process which stops at p_k . Hence, no more sites in B should be examined further and the algorithm stops correctly.

4 Experiments

In this section, we show our experimental results. To evaluate the proposed protocol, we measure the tuning time and latency. In the experiments, the point data are generated uniformly in the area of $[0, 1000] \times [0, 1000]$. For the queries, the query point is generated randomly in the given area and the value of k varies from 1 to 51. The broadcast schedules are generated randomly. The data reported is the average of 100 different queries for each value of k on a given broadcast. We use one site (or packet) as a measure unit for simplicity. Recall that, a naive approach for the kNN search is to always listen to the broadcast to derive the results. Such an approach will cost 1000 (number of sites) on the tuning time and latency. In the following, we first show the result on the tuning time.

4.1 Tuning Time

The tuning time indicates the cost spent on listening to the broadcast by clients and reflects the power consumption of the mobile clients for the k NN search in wireless broadcast environments. Our k NN search starts at the arbitrary position of a broadcast. Recall that the proposed protocol uses D -list to record the positions, so the query processing can skip listening some packets. Hence, our protocol can avoid spending energy on receiving the irrelevant sites. The experimental result shown in Figure 5 indicates that our protocol can explore fewer data points than the naive approach mentioned above, thus leading to a less tuning time. The tuning time increases as the value of k becomes larger and becomes flat when k is larger than 35. So, our approach is more effective when k is small in terms of tuning time. All of our experimental results have the similar trend.

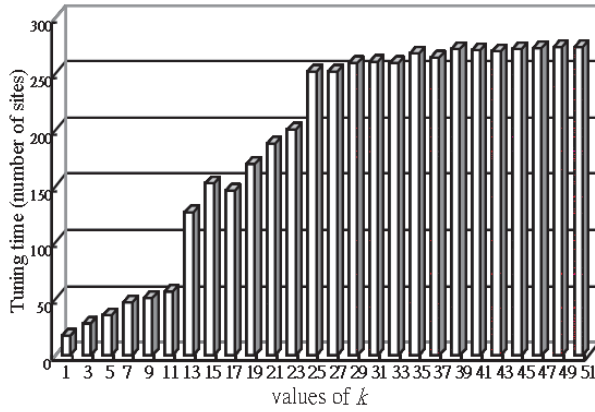


Figure 5: The tuning time of the proposed k NN search process on the data set of size 1,000 with different values of k .

4.2 Access Latency

We now discuss the access latency which is the time elapsed from requesting to receiving the desired data points. The access latency indicates the quality of service(QoS). In our protocol, we use C -list to store the candidate sites. Using the candidate sites, we provide a condition to derive the final result and allow the process can stop earlier than the naive approach. Hence, our proposed protocol can achieve a shorter latency. Figure 6 demonstrates one of experimental results about the latency. All the other experimental results have the same trend. The latency increases as the value of k increases. When k is small, the protocol will experience a shorter latency. When k is larger than 27,

all the k NN queries result in a similar latency, that is smaller than the half of the latency, 1,000 number of packets, experiences by the naive approach.

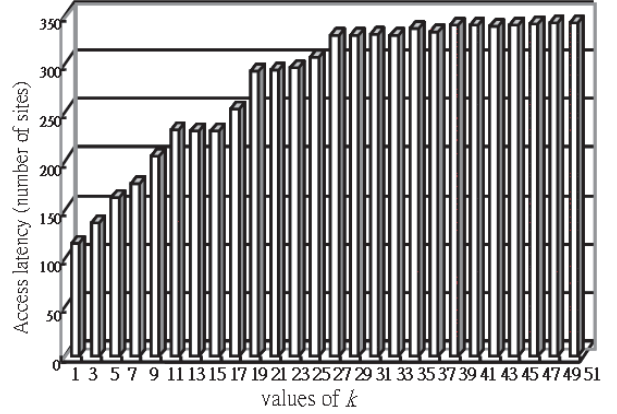


Figure 6: The latency of the proposed k NN search process on the data set of size 1,000 with different values of k .

5 Conclusions

In this paper, we present a k NN search protocol in wireless broadcast environments. The idea of the proposed protocol is simple but can reduce the latency and tuning time effectively. Unlike some other protocols, the protocol does not use an index in the broadcast and can support the k NN search for an arbitrary k . Our experimental results show the performance of our approach in terms of tuning time and latency. In the future, we plan to have more simulation experiments on our protocol and compare it with the other existing k NN search protocols in wireless broadcast environment. We will also discuss further about the constraints on the distribution of the input data points on the plane. It is possible that a data point u in some given data point set of size n may have $n - 1$ 1-order neighbors. This will hurt the proposed search protocol due to the limited packet size. However, this case is rare in general. Besides, we believe that the proposed approach can be easily adapted in the traditional disk-based environment. On the disk-based environments, our approach does not need an index structure. How to place the data points into the pages in order to make the search more efficient in terms of number of I/O's becomes interesting.

References

- [1] Swarup Acharya, Rafael Alonso, Michael Franklin, and Stanley Zdonik. Broadcast disks: data management for asymmetric communication environments. In *Proceedings of the 1995 ACM SIGMOD international conference on management of data*, pages 199–210, 1995.
- [2] Swarup Acharya, Michael Franklin, and Stanley Zdonik. Balancing push and pull for data broadcast. In *Proceedings of the 1997 ACM SIGMOD international conference on management of data*, pages 183–194, 1997.
- [3] S. Berchtold, D. A. Keim, H.-P. Kriegel, and T. Seidl. Indexing the solution space: A new technique for nearest neighbor search in high-dimensional space. *IEEE Transactions on Knowledge and Data Engineering*, 12(1):45–57, 2000.
- [4] Y. C. Chehadeh, A. R. Hurson, and M. Kavehrad. Object organization on a single broadcast channel in the mobile computing environment. *Multi-media Tools and Applications*, 9(1):69–94, 1999.
- [5] M. de Berg, M. van Krefeld, M. Overmars, and O. Schwarzkopf. *Computational Geometry: Algorithms and Applications*. Springer, 2000.
- [6] Bugra Gedik, Aameek Singh, and Ling Liu. Energy efficient exact knn search in wireless broadcast environments. In *Proceedings of the 12th annual ACM international workshop on geographic information systems*, pages 137–146, 2004.
- [7] Susanne Hambrusch, Chuan-Ming Liu, Walid G. Aref, and Sunil Prabhakar. Efficient query execution on broadcasted index tree structures. *Data and Knowledge Engineering*, 60(3):511–529, 2007.
- [8] Susanne Hambrusch, Chuan-Ming Liu, and Sunil Prabhakar. Broadcasting and querying multi-dimensional index trees in a multi-channel environment. *Information Systems*, 31(8):870–886, 2006.
- [9] Susanne E. Hambrusch, Chuan-Ming Liu, Walid G. Aref, and Sunil Prabhakar. Query processing in broadcasted spatial index trees. In *Proceedings of the 7th international symposium on advances in spatial and temporal databases*, volume 2121, pages 502–510. Springer-Verlag, 2001.
- [10] Sohail Hameed and Nitin H. Vaidya. Efficient algorithms for scheduling data broadcast. *ACM/Baltzer Journal of Wireless Networks*, 5(3):183–193, 1999.
- [11] Hao-Ping Hung and Ming-Syan Chen. On exploring channel allocation in the diverse data broadcasting environment. In *Proceedings of the 25th IEEE International Conference on Distributed Computing Systems (ICDCS'05)*, pages 729–738, 2005.
- [12] A.R. Hurson and Y. Jiao. Data broadcasting in a mobile environment. In *Wireless Information Highway*, chapter 4, pages 96–154. IRM Press, 2004.
- [13] T. Imieliński, S. Viswanathan, and B. R. Badrinath. Data on air: Organization and access. *IEEE Transactions on Knowledge and Data Engineering*, 9(3):353–372, 1997.
- [14] S. Khanna and S. Zhou. On indexed data broadcast. *Journal Computer and System Sciences*, 60:575–591, 2000.
- [15] Mohammad Kolahdouzan and Cyrus Shahabi. Voronoi-based k nearest neighbor search for spatial network databases. In *vldb'2004: Proceedings of the Thirtieth international conference on Very large data bases*, pages 840–851. VLDB Endowment, 2004.
- [16] D.-T. Lee. On k -nearest neighbor voronoi diagrams in the plane. *IEEE Transactions on Computers*, 31(6):478–487, 1982.
- [17] G. Lee, S.-C. Lo, and A.L.P. Chen. Data allocation on wireless broadcast channels for efficient query processing. *IEEE Transactions on Computers*, 51(10), 2002.
- [18] Chuan-Ming Liu. *Broadcasting and blocking large data sets with an index tree*. PhD thesis, Purdue University, West Lafayette, IN, 2002.
- [19] Chuan-Ming Liu and Shu-Yu Fu. Effective protocols for knn search on broadcast multi-dimensional index trees. *Information Systems*, 33(1):18–35, 2008.
- [20] Chuan-Ming Liu and Kun-Feng Lin. Disseminating dependent data in wireless broadcast environments. *Distributed and Parallel Databases*, (1):1–25, 2007.
- [21] N. Roussopoulos, S. Kelley, and F. Vincent. Nearest neighbor queries. In *Proceedings of the*

1995 ACM SIGMOD International Conference on Management of Data, San Jose, California, May 22-25, 1995, pages 71–79, 1995.

- [22] N. Vaidya S. Hameed. Efficient algorithms for scheduling data broadcast. *ACM/Baltzer Journal of Wireless Networks*, 5(3):183–193, 1999.
- [23] H. Samet. *Applications of Spatial Data Structures: Computer Graphics, Image Processing, and GIS*. Addison-Wesley, Reading, Massachusetts, 1990.
- [24] H. Samet. *The Design and Analysis of Spatial Data Structures*. Addison-Wesley, Reading, Massachusetts, 1990.
- [25] C.-J. Su, L. Tassiulas, and V. J. Tsotras. Broadcast scheduling for information distribution. *ACM/Baltzer Journal of Wireless Networks*, 5(2):137–147, 1999.
- [26] K.L. Tan, J. X. Yu, and P.K. Eng. Supporting range queries in a wireless environment with nonuniform broadcast. *Data Knowledge Engineering*, 29(2):201–221, 1999.
- [27] Baihua Zheng, Xia Wu, Xing Jin, and Dik Lun Lee. Tosa: a near-optimal scheduling algorithm for multi-channel data broadcast. In *Proceedings of the 6th international conference on Mobile data management*, pages 29–37, 2005.
- [28] Baihua Zheng, Jianliang Xu, Wang-Chien Lee, and Dik Lun Lee. Grid-partition index: a hybrid method for nearest-neighbor queries in wireless location-based services. *The VLDB Journal*, 15(1):21–39, 2006.